# OS-Symphony: A Holistic Framework for Robust and Generalist Computer-Using Agent

Bowen Yang, Kaiming Jin, Zhenyu Wu, Zhaoyang Liu, Qiushi Sun, et al.

Paper version: arXiv 2601.07779v1 · CUA Radar Reel poster, not made or endorsed by the authors

## Problem

> *current frameworks struggle with robustness in long-horizon workflows and generalization in novel domains*

History curation loses screenshots; retrieval is text-only, offline, or costly to keep fresh. The paper answers with milestone memory and live visual search.

## Motivation: two gaps

1 **Visual loss.** Coarse history pruning; drift and loops slip by.

2 **Blind retrieval.** Text-only RAG misses visual cues; local KBs age fast.

## Evaluation setup

OSWorld · 361 tasks · WindowsAgentArena · 154 · MacOSArena · 63 · GPT-5 / GPT-5-Mini · Qwen3-VL 8B-32B · grounder UI-TARS-1.5-7B · 50 / 100-step cap

- **All online**, desktop; no offline metrics.
- **Baselines:** Agent S3 reproduced by the authors; rest from leaderboards.

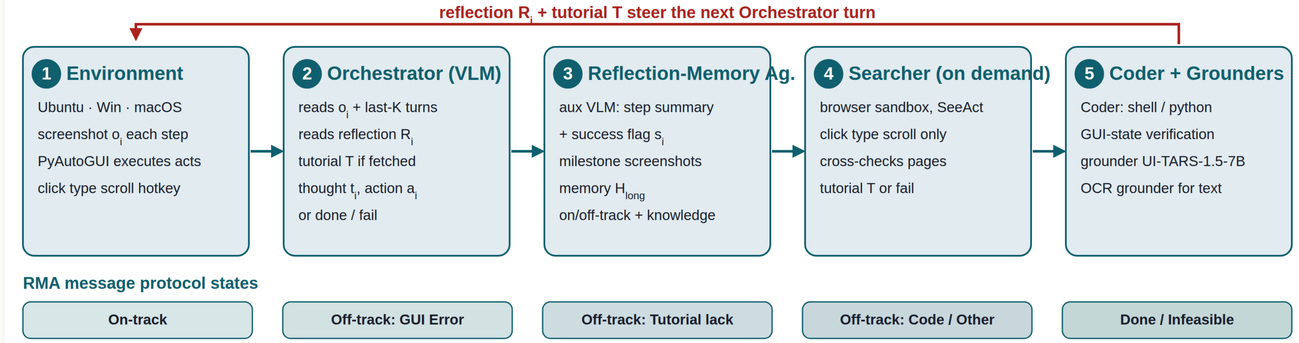
## Our read (CUA Radar)

| 61 | ★★ | online |
|---|---|---|
| score of 100 | band | evidence level |

- **Reports:** online eval; reproduced Agent S3; cost; ablation.
- **Flags:** no held-out split; no seeds/variance; RMA verdicts unvalidated.
- Pattern C18. Authors' numbers, not rerun.

## Method: one Orchestrator, reflection memory, tool agents

- **Sees / acts:** screenshot in; click, type, scroll, hotkey out via PyAutoGUI; plus calls to Searcher and Coder; Linux, Windows, macOS.
- **RMA:** aux VLM summarizes and verifies each step; milestone-gated long-term memory; on/off-track reflection with 4 error types.
- **Searcher:** See-Act browsing in a sandbox; returns a tutorial or fail; rule-based loop detection (pHash + SSIM) triggers it.

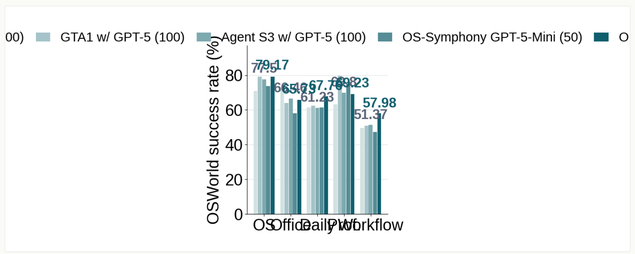



*Our redraw (not the paper's figure): the Orchestrator loop with the RMA and tool agents, from the method text.*

$$t_i,\ a_i = F_O(I,\ R_i,\ o_i,\ T,\ H^{\text{short}})$$

*Orchestrator step: instruction, reflection, screenshot, tutorial, last-K memory (Eq. 1)*

## Key results: online success (%)




*Tables 1, 2, 11 redrawn; 50-step runs unless noted; single runs, no variance reported.*

**OSWorld 65.84 w/ GPT-5 (100-step) vs 62.63 Agent S3**

**SO WHAT →** *WindowsAgentArena 63.5 · MacOSArena 46.03, both reported SOTA.*

## Ablation: OSWorld Avg (%)

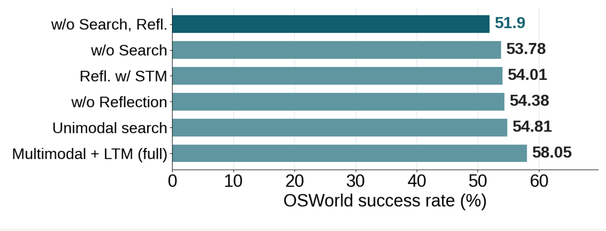



*Table 3 redrawn: GPT-5-Mini + UI-TARS-1.5-7B, 50 steps.*

**SO WHAT →** *Base (no search, no reflection) 51.90; search and reflection each add ~4 pts. Unimodal search 54.81 vs multimodal 58.05; Daily-domain gain +22.1% rel. over no search. Naive last-K reflection (54.01) trails no reflection (54.38): a bad reflection is worse than none. w/o RMA costs +3.3 steps per task; the full run uses 900k tokens, 15.2 steps vs 18.4 for the base.*

## Headline Numbers

**65.84%**
**OSWorld success rate**
*GPT-5, 100 steps; 63.61% at 50*

**63.5%** WindowsAgentArena
**46.03%** MacOSArena
**79.40%** Pass@5

## The paper's own controls

| CONTROL (OSWORLD) | OS-SYMPHONY | COMPARED WITH |
|---|---|---|
| w/o Coder (50-step) | 57.42 | 63.61 with Coder |
| Grounder swap | 46.86 UI-TARS-1.5-7B | GTA1-32B 46.32 |
| VLM cost, Workflow | GPT-5-Mini 47.37 ≈$30 | GPT-5 54.86 ≈$150 |
| RMA GUI-Error audit | ~90 of 100 correct | ~10 hallucinated |
| Pass@2 (GPT-5, 100-step) | 74.14% | human 72.4% |

Agent S3 rerun by the authors at matched budgets

## Takeaway

**Milestone memory + on-demand visual search lift one VLM across three desktop OSes; gains, ablations and cost reported, single runs only.**

Pass@2 74.14% > human · 8-image context cap · ~270k tokens per task · search used 35x (0.6%)

## Scan to Read

Paper · Code